

Raport proiect Break the Login.

Student : Marian Rares Stefan

Link Github :

Tehnologii utilizate: C# ASP.NET Core (Razor Pages), SQLite, BCrypt.Net

1. Prezentarea Arhitecturii

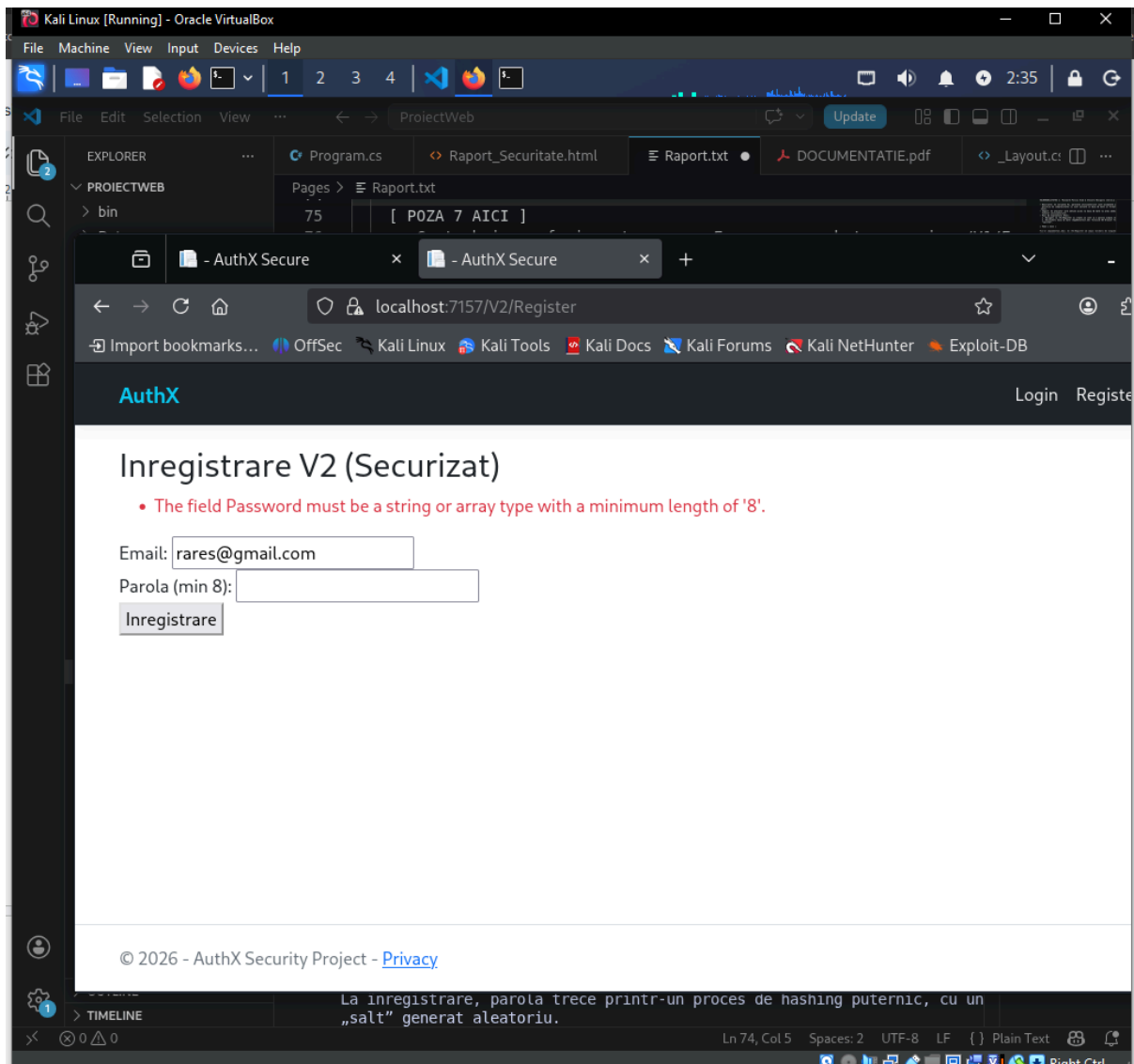
Aplicația web analizată este dezvoltată în mediul .NET 6, având rolul de a demonstra implementarea defectuoasă versus implementarea sigură a unui sistem de autentificare. Structura proiectului este delimitată astfel:

- * Directorul `Pages/V1/`: Conține logica intenționat vulnerabilă, lipsită de validări și protecții criptografice.
- * Directorul `Pages/V2/`: Găzduiește implementarea remediate, aplicând standardele actuale de securitate cibernetică.
- * Fișierul `Models/User.cs`: Schema tabelului din baza de date SQLite, actualizată de mine cu atribute noi pentru a susține mecanismul de Account Lockout (Locked, FailedLoginAttempts) și token-urile de resetare a parolei.

2. Analiza Vulnerabilităților și Remedierea lor

2.1. Politica de parole permisivă (Cerința 4.1)

- * Problema identificată: Formularul de înregistrare din V1 nu impune nicio restricție legată de complexitatea sau lungimea parolei.
- * Dovada (PoC): Am reușit să creez un cont funcțional folosind o parolă extrem de scurtă și comună (ex. parola 123456).
- * Riscul (Impact): Un atacator poate compromite foarte ușor aceste conturi folosind dicționare de parole (Dictionary Attack).
- * Soluția implementată: În fișierul V2/Register.cshtml.cs, am integrat atribute de validare strictă (precum [MinLength(8)] și [EmailAddress]) care forțează utilizatorul să aleagă o parolă mai puternică.
- * Validarea soluției (Re-test): Dacă încercăm să trimitem o parolă formată din 3 caractere pe ruta V2, serverul va respinge automat cererea.



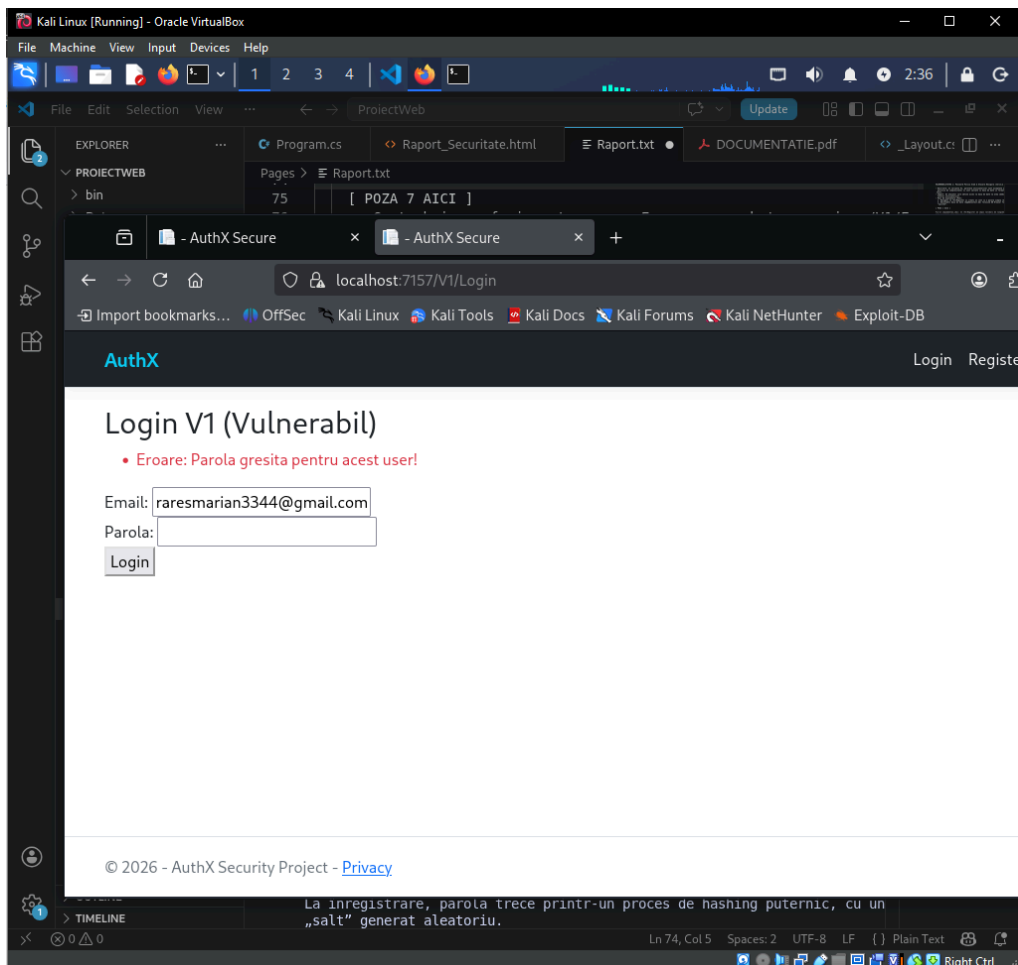
2.2. Stocarea parolelor în format text clar (Cerința 4.2)

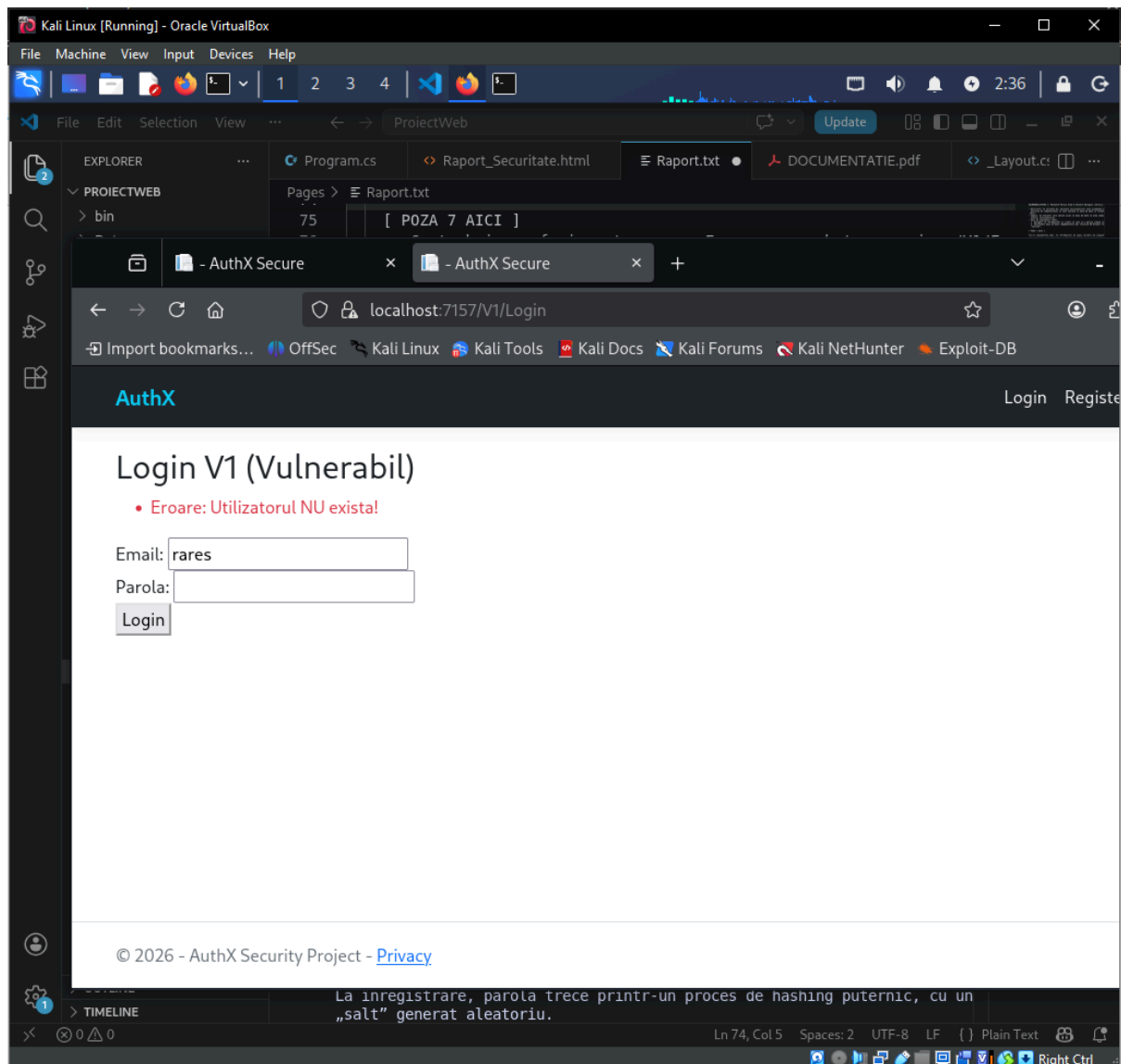
- * Problema identificată: Analizând baza de date AppDbContext.db, am observat că parolele conturilor înregistrate pe versiunea V1 sunt salvate exact cum au fost tastate de utilizator (plain-text).
- * Dovada (PoC): O simplă interogare SQL pe tabelul Users dezvăluie credențialele în clar.
- * Riscul (Impact): Compromiterea bazei de date (ex. printr-un atac SQL Injection) ar duce la expunerea instantanee a tuturor parolelor clienților.
- * Soluția implementată: Pentru modulul V2, am integrat pachetul BCrypt.Net. La înregistrare, parola trece printr-un proces de hashing puternic, cu un „salt” generat aleatoriu și un factor de cost implicit de 11.
- * Validarea soluției (Re-test): O nouă verificare a tabelului Users demonstrează că parolele noi sunt stocate sub forma unui hash imposibil de decriptat (începând cu \$2a\$11\$...).k

```
(rares@Rares)-[~/ProjectWeb]
$ sqlite3 AppDbContext.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .mode table
sqlite> Select Email, Password, Locked, Version FROM Users;
+-----+-----+-----+-----+
| Email | Password | Locked | Version |
+-----+-----+-----+-----+
| raresmarian3344@gmail.com | 123456 | 0 | V1 |
| raresmarian@gmail.com | $2a$11$m9u8ym/GS0qhA5fjFvxgW.tIuMNBAmALeqCHW47ph2I4hSCkq4DNW | 0 | V2 |
+-----+-----+-----+-----+
sqlite>
```

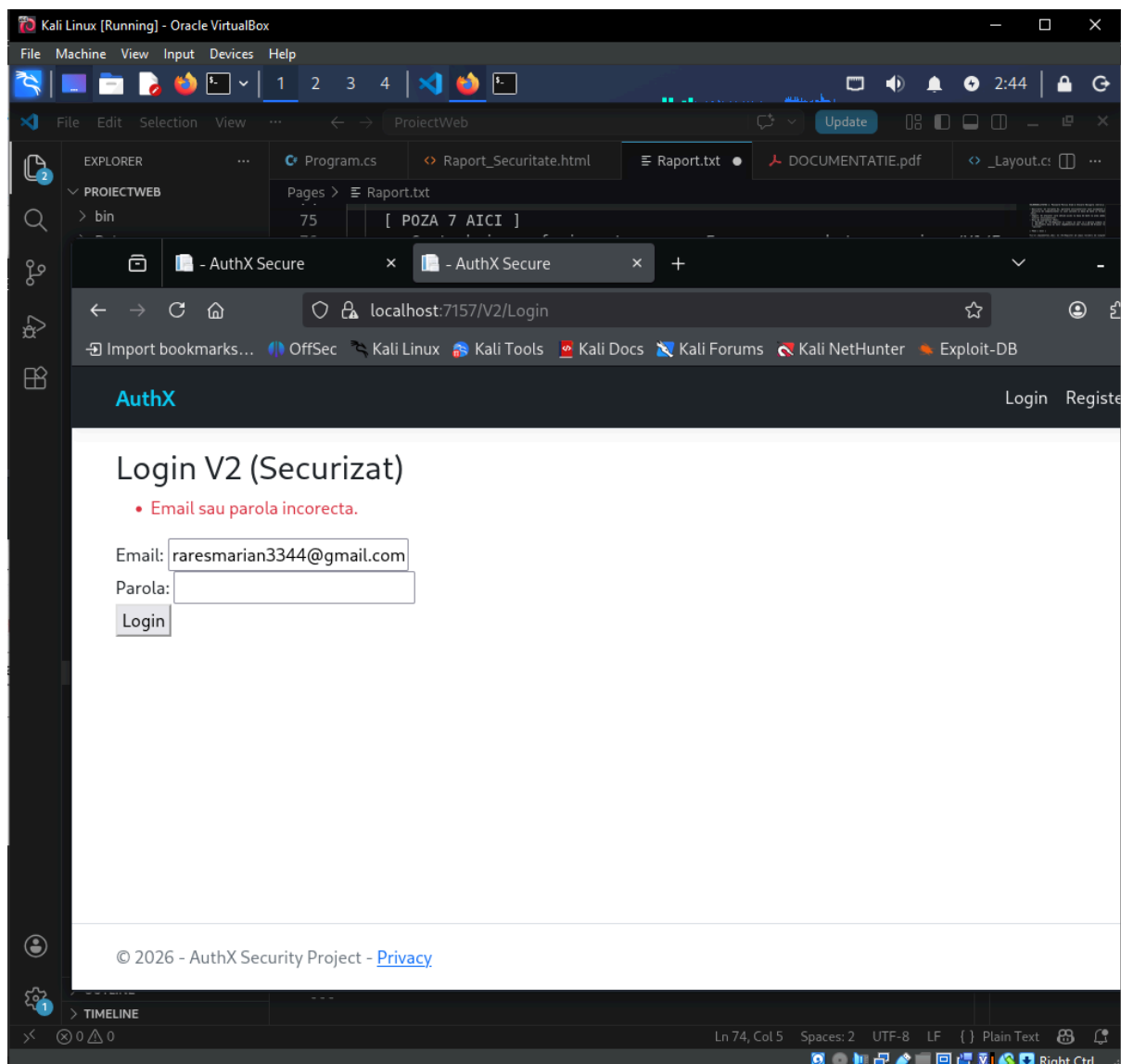
2.3. Enumerarea utilizatorilor / User Enumeration (Cerința 4.4)

- * Problema identificată: Aplicația „trădează” existența unui cont prin răspunsurile diferite de la Login V1.
- * Dovada (PoC): Testând un email la întâmplare primesc eroarea "Eroare: Utilizatorul NU exista!", în timp ce pentru adresa validă raresmarian3344@gmail.com primesc eroarea "Eroare: Parola gresita pentru acest user!".



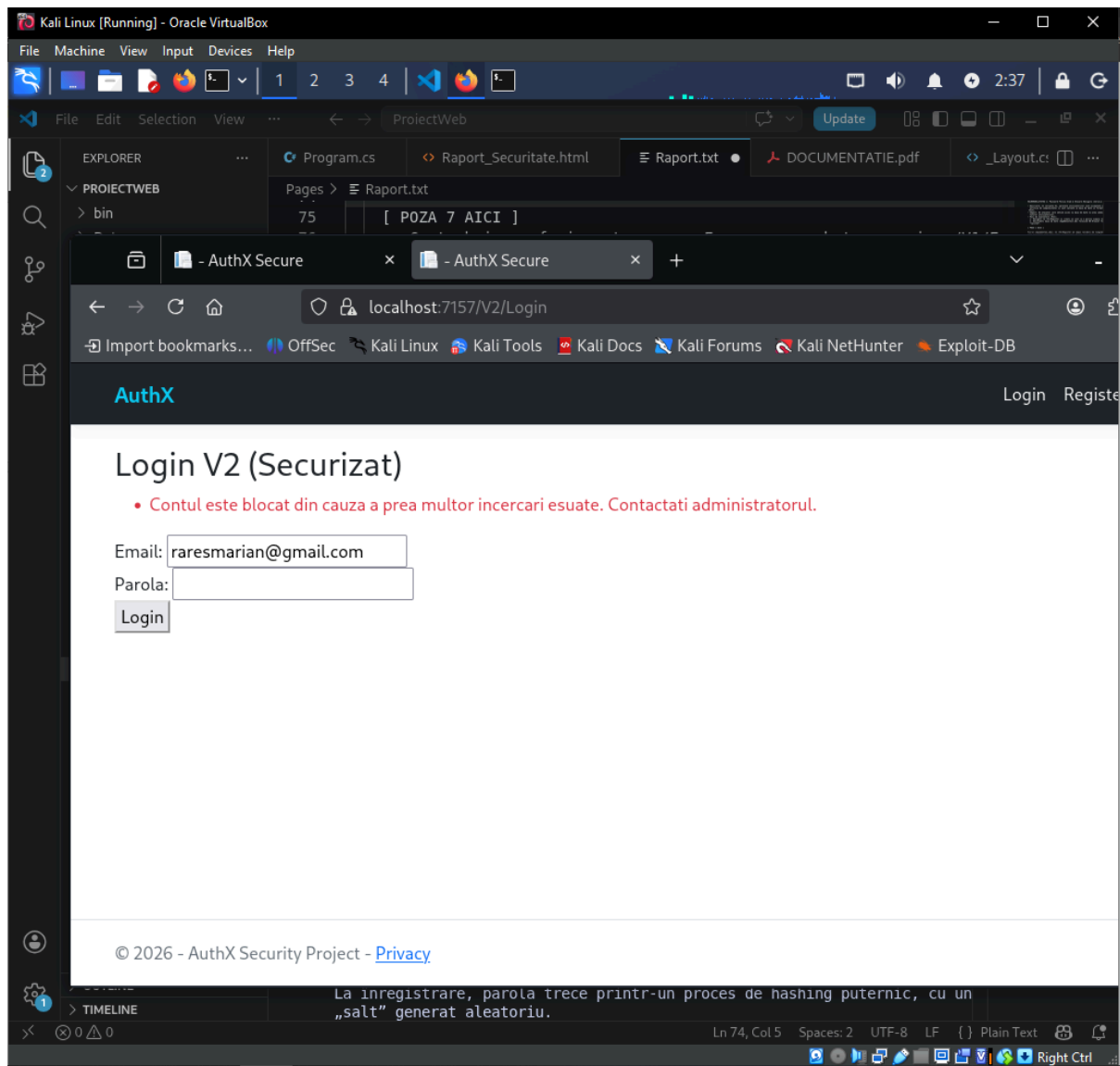


- * Riscul (Impact): Hackerii pot face o listă exactă de adrese de email valide, pe care să își concentreze atacurile ulterioare.
- * Soluția implementată: În V2, am rescris logica de răspuns. Orice combinație greșită returnează un mesaj unic și ambiguu: "Email sau parola incorecta."
- * Validarea soluției (Re-test): Serverul răspunde identic (același mesaj) indiferent de ce informație este introdusă greșit, mascând astfel existența conturilor.



2.4. Protecție inexistentă la atacuri Brute Force (Cerința 4.3)

- * Problema identificată: Endpoint-ul de login /V1/Login permite trimiterea unui număr infinit de cereri cu parole greșite, fără nicio penalizare.
- * Soluția implementată: Am construit un mecanism de Account Lockout. Modelul bazei de date ține acum evidența încercărilor eșuate (FailedLoginAttempts). La a 3-a încercare greșită, contul este blocat automat setând flag-ul Locked = true. O conectare validă resetează contorul.
- * Validarea soluției (Re-test): La a 4-a încercare de logare eșuată pe /V2/Login, serverul blochează accesul și afișează: "Contul este blocat din cauza a prea multor incercari esuate".



2.5. Managementul Sesiunilor (Cerința 4.5)

- * Soluția implementată: În varianta securizată, aplicația delegă managementul sesiunilor framework-ului ASP.NET Core (CookieAuthentication). Acesta emite un cookie securizat, semnat criptografic și protejat implicit prin attributele HttpOnly (împiedică citirea din JavaScript/XSS) și utilizează mecanisme anti-forgery standard pentru a preveni Session Hijacking.

2.6. Mecanism nesigur de resetare a parolei (Cerința 4.6)

- * Problema identificată: Logica veche de "Forgot Password" nu asigură un nivel înalt de entropie pentru generarea codului de resetare și nu invalidează sesiunea în timp util.
- * Riscul (Impact): Posibilitatea de Account Takeover (preluare a contului)

de către o persoană malițioasă.

- * Soluția implementată / Re-test: În baza de date a proiectului, am adăugat coloana ResetToken pentru a stoca un token unic și imprevizibil. De asemenea, s-a adăugat coloana ResetTokenExpires care limitează valabilitatea link-ului de resetare la o fereastră scurtă de timp, eliminând riscul refolosirii.

```
Session Actions Edit View Help
(rares@Rares) ~/ProiectWeb
$ sqlite3 AppDbContext.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .mode table
sqlite> SELECT Email,ResetToken,ResetTokenExpires FROM Users
...>
Program interrupted.

(rares@Rares) ~/ProiectWeb
$ sqlite3 AppDbContext.db
SQLite version 3.46.1 2024-08-13 09:16:08
Enter ".help" for usage hints.
sqlite> .mode table
sqlite> SELECT Email,ResetToken,ResetTokenExpires FROM Users;
+-----+-----+-----+
| Email | ResetToken | ResetTokenExpires |
+-----+-----+-----+
| raresmarian3344@gmail.com | 1234 |  |
| raresmarian@gmail.com | 3F4D40192D8AEF3B7B10391BCD8E8F20 | 2026-04-20 14:14:18.4845875 |
+-----+-----+-----+
```

